

Autonomous Finance

Security Assessment

March 4th, 2025 — Prepared by OtterSec

Akash Gurugunti sud0u53r.ak@osec.io

Nicola Vella nick0ve@osec.io

Robert Chen r@osec.io

Table of Contents

Executive Summary **2**

Overview 2

Key Findings 2

Scope **3**

Findings **4**

Vulnerabilities **5**

OS-ATF-ADV-00 | DOS Due to Large Value of Excess Tokens 6

OS-ATF-ADV-01 | Locked Tokens in the Vault Token Account 8

OS-ATF-ADV-02 | Inaccurate Data Due to Epoch Mismatch 9

General Findings **11**

OS-ATF-SUG-00 | Incorrect Accounting in Withdraw SOL Functionality 12

OS-ATF-SUG-01 | Inconsistencies in Eject User Instruction 14

OS-ATF-SUG-02 | Code Maturity 15

OS-ATF-SUG-03 | Code Redundancy 16

Appendices

Vulnerability Rating Scale **17**

Procedure **18**

© 2025 Otter Audits LLC. All Rights Reserved.

01 — Executive Summary

Overview

Autonomous Finance engaged OtterSec to assess the `stake-vault` program. This assessment was conducted between January 10th, 2024 and February 14th, 2025. For more information on our auditing methodology, refer to Appendix B.

Key Findings

We produced 7 findings throughout this audit engagement.

In particular, we identified a vulnerability concerning the possibility of an underflow if the value of excess tokens exceeds that of the minted tokens, either due to direct transfers to the vault or a reduction in the active `SOL` deposits without adjusting the minted token balance (OS-ATF-ADV-00). Additionally, if the last update epoch in stake pool does not match the current epoch, it may result in an inaccurate total lamports value, affecting pool token pricing calculations (OS-ATF-ADV-02).

Furthermore, the vault incorrectly treats the full deposit amount (including fees) as part of the users' claimable balance, inflating the vault's token balance and rendering the locked funds unclaimable by both users and the vault (OS-ATF-ADV-01).

We also made recommendations regarding the need for improved validation and consistency checks (OS-ATF-SUG-01) and suggested removing unnecessary and redundant instances (OS-ATF-SUG-03). Moreover, we advised adhering to coding best practices (OS-ATF-SUG-02).

© 2025 Otter Audits LLC. All Rights Reserved.

02 — Scope

The source code was delivered to us in a Git repository at https://github.com/Autonomous-Finance/stake-vault. This audit was performed against commit 35d7bb2.

A brief description of the program is as follows:

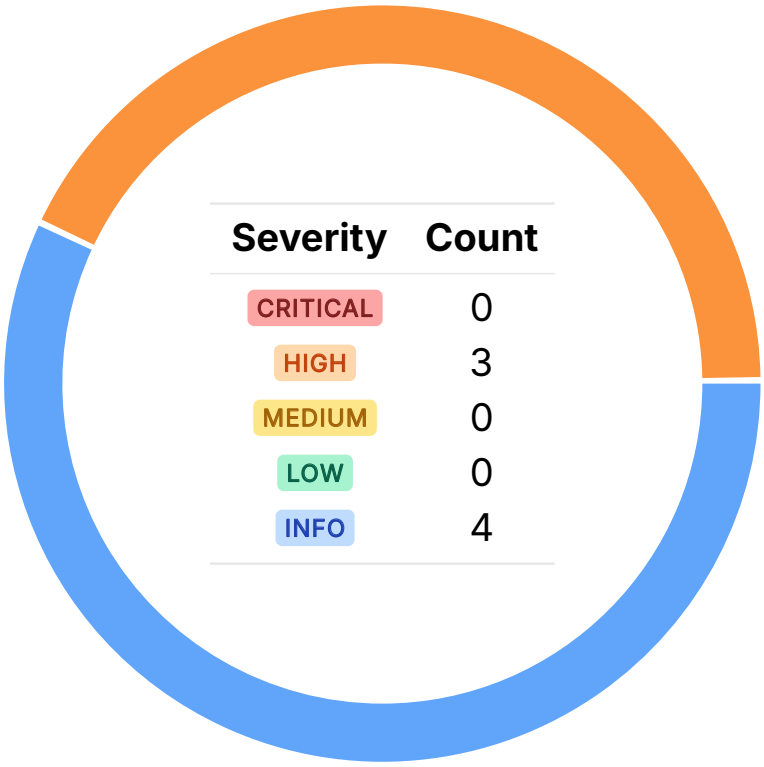
Name	Description
stake-vault	The Solana Staking Vault Bridge acts as a decentralized staking vault that enhances liquidity by integrating Jito's liquid staking pool. While it offers permissionless staking and withdrawals, reward harvesting remains centralized, ensuring vault sustainability but reducing immediate reward access for depositors.

© 2025 Otter Audits LLC. All Rights Reserved.

03 — Findings

Overall, we reported 7 findings.

We split the findings into **vulnerabilities** and **general findings**. Vulnerabilities have an immediate impact and should be remediated as soon as possible. General findings do not have an immediate impact but will aid in mitigating future vulnerabilities.



Severity	Count
CRITICAL	0
HIGH	3
MEDIUM	0
LOW	0
INFO	4

© 2025 Otter Audits LLC. All Rights Reserved.

04 — Vulnerabilities

Here, we present a technical analysis of the vulnerabilities we identified during our audit. These vulnerabilities have *immediate* security implications, and we recommend remediation as soon as possible.

Rating criteria can be found in Appendix A.

ID	Severity	Status	Description
OS-ATF-ADV-00	HIGH	RESOLVED	An underflow may occur if excess tokens exceed minted tokens, either due to direct transfers to the vault or a reduction in `active_sol_deposits` without a corresponding adjustment to the minted token balance.
OS-ATF-ADV-01	HIGH	RESOLVED	The vault incorrectly treats the full deposit amount (including fees) as part of the users' claimable balance, inflating the vault's token balance and rendering the locked funds unclaimable by both the users and the vault.
OS-ATF-ADV-02	HIGH	RESOLVED	If the `last_update_epoch` in `StakePool` does not match the current epoch, it may result in an inaccurate `total_lamports` value, affecting pool token pricing calculations.

© 2025 Otter Audits LLC. All Rights Reserved.

DOS Due to Large Value of Excess Tokens HIGH OS-ATF-ADV-00

Description

`distribute::distribute_inner` contains a potential denial-of-service (DoS) vulnerability due to a mismatch between `vault_state.minted_tokens` and the actual balance in `vault_token_account`. If a user manually transfers directly into `vault_token_account`, the vault's balance increases, and `excess_tokens` also increases more than expected. Since `vault_state.minted_tokens` is not updated when tokens are externally deposited, it remains the same. As a result, `excess_tokens` will be greater than `vault_state.minted_tokens`.

Consequently, when updating the vault's minted tokens and resetting the staged amount, `vault_state.minted_tokens.checked_sub(excess_tokens)` will underflow. When the function reverts due to the underflow, reward distribution will be prevented.

>_ stake-vault/src/instructions/distribute.rs RUST

```rust
pub fn distribute_inner(ctx: Context<Distribute>) -> Result<()> {
    [...]
    // Update the vault's minted tokens and reset the staged amount
    vault_state.minted_tokens = vault_state
        .minted_tokens
        .checked_sub(excess_tokens)
        .ok_or(CalculationErrorType::ArithmeticUnderflow)?;
    [...]
    Ok(())
}
```

An underflow may also be triggered by transferring lamports to `vault_state`. When `SOL` is withdrawn, excess lamports from `vault_state` are deducted from `vault_state.active_sol_deposits`. If lamports are transferred into `vault_state`, this does not increase `minted_tokens` but reduces `tokens_needed_to_cover_deposits`, increasing `excess_tokens` and resulting in the possibility of DoS via underflow.

This DoS is temporary, and on depositing more `SOL`, `vault_state.active_sol_deposits` increases, raising `tokens_needed_to_cover_deposits`, which reduces `excess_tokens` and proportionally increases `minted_tokens`. Nevertheless, it creates an inconsistency between `vault_state.minted_tokens` and the actual vault ATA balance.

Remediation

Rework the tracking logic for `vault_state.minted_tokens`.

© 2025 Otter Audits LLC. All Rights Reserved.

Patch

Fixed in e73bdc0.

© 2025 Otter Audits LLC. All Rights Reserved.

Locked Tokens in the Vault Token Account HIGH OS-ATF-ADV-01

Description

The vulnerability arises arises because `vault_state.active_sol_deposits` and `user_account.deposited_lamports` track different values with respect to deposit amounts and fees, resulting in an unintended token lock. In the `Distribute` instruction, `vault_state.active_sol_deposits` is always greater than the sum of all `user_account.deposited_lamports`.

This occurs because, in `DepositSol` instruction, `user_account.deposited_lamports` is increased by (lamports deposited - fee), while the full amount deposited (before any fee deduction) is added to `vault_state.active_sol_deposits`, which tracks the total deposits across all users, regardless of fees. Thus, `vault_state.active_sol_deposits` is updated with the full deposit amount, while `user_account.deposited_lamports` is updated only after deducting the fee. Consequently, the token equivalent of manager and referrer fees is locked in the vault token account indefinitely, unclaimable by both the users and the vault authority.

Remediation

Modify the logic to track the fees deducted from the deposits, allowing the vault to calculate the actual deposits and the fees.

Patch

Fixed in d914c92.

© 2025 Otter Audits LLC. All Rights Reserved.

Inaccurate Data Due to Epoch Mismatch HIGH OS-ATF-ADV-02

Description

The `total_lamports` field in `StakePool` represents the total amount of `SOL` (in lamports) staked in the pool. The issue arises because this field may not be fully up-to-date if the `last_update_epoch` does not match the current epoch. The `total_lamports` field needs to reflect the total stake that has been updated after each epoch. If the `last_update_epoch` is outdated (it does not match the current epoch), the `total_lamports` value may be stale. Consequently, the calculation of pool token price, which relies on `total_lamports` will be inaccurate.

>_ stake-vault/src/state/stake_pool.rs RUST

```rust
pub fn calc_pool_tokens_required_for_withdrawal(
    pool: &SplStakePool,
    desired_lamports: u64,
) -> Result<u64> {
    if pool.total_lamports == 0 || pool.pool_token_supply == 0 {
        return Err(SplStakePoolErrorType::StakePoolError.into());
    }

    let numerator = (desired_lamports as u128)
        .checked_mul(pool.pool_token_supply as u128)
        .ok_or(CalculationErrorType::ArithmeticOverflow)?;

    let denominator = pool.total_lamports as u128;

    let pool_tokens = numerator
        .checked_add(denominator - 1)
        .ok_or(CalculationErrorType::ArithmeticOverflow)?
        .checked_div(denominator)
        .ok_or(CalculationErrorType::ZeroDivision)?;

    Ok(u64::try_from(pool_tokens).map_err(|_| CalculationErrorType::ArithmeticOverflow)?)
}
```

Remediation

Check `last_update_epoch` against the current epoch before using `StakePool` data to ensure `total_lamports` is accurate for computing pool token prices in `ViewAvailableExcess` and `Distribute`.

© 2025 Otter Audits LLC. All Rights Reserved.

Patch

Fixed in 1bbd89b.

© 2025 Otter Audits LLC. All Rights Reserved.

05 — General Findings

Here, we present a discussion of general findings during our audit. While these findings do not present an immediate security impact, they represent anti-patterns and may result in security issues in the future.

ID	Description
OS-ATF-SUG-00	`WithdrawSol` incorrectly updates deposited tokens when withdrawals are partially covered by excess lamports, resulting in improper accounting.
OS-ATF-SUG-01	`EjectUser` may not be fulfilled if the user's lamports are less than the required rent minimum, requiring the protocol to gift additional lamports to ban the user. Also, the instruction does not explicitly verify if a user has already been ejected.
OS-ATF-SUG-02	Suggestions regarding inconsistencies in the codebase and ensuring adherence to coding best practices.
OS-ATF-SUG-03	The codebase contains multiple cases of redundant code that should be removed for better maintainability and clarity.

© 2025 Otter Audits LLC. All Rights Reserved.

Incorrect Accounting in Withdraw SOL Functionality OS-ATF-SUG-00

Description

`WithdrawSol` does not correctly account for the scenario where `desired_lamports` are partially covered by excess lamports in `vault_state`. When `desired_lamports_after_excess == 0`, it implies that the entire withdrawal amount is covered by excess lamports in `vault_state`, so no staking pool tokens (LP tokens) need to be burned. As a result, the `if desired_lamports_after_excess > 0` conditional block does not execute, and consequently, `required_tokens` and `fee_paid` will be zero.

>_ stake-vault/src/instructions/withdraw_sol.rs RUST

```rust
pub fn withdraw_sol_inner(ctx: Context<WithdrawSol>, desired_lamports: u64) -> Result<()> {
    [...]
    let mut required_tokens = 0;
    let mut fee_paid = 0;

    if desired_lamports_after_excess > 0 {
        // If there are still lamport balances left that we have to unstake for, do so
        required_tokens =
            calc_pool_tokens_required_for_withdrawal(&stake_pool,
                ↪  desired_lamports_after_excess)?;
        [...]
        require!(
            lamports_received == expected_lamports_received,
            VaultErrorType::InsufficientSolReceived
        );
    }
    **ctx
        .accounts
        .vault_state
        .to_account_info()
        .try_borrow_mut_lamports()? -= desired_lamports.saturating_sub(fee_paid);

    **ctx
        .accounts
        .beneficiary
        .to_account_info()
        .try_borrow_mut_lamports()? += desired_lamports.saturating_sub(fee_paid);
    [...]
}
```

The `desired_lamports` are directly subtracted from `vault_state` and sent to the beneficiary, and while `deposited_lamports` is updated correctly, `deposited_tokens` is not. Since `deposited_tokens` is not reduced, the user's ownership share in the staking pool remains unchanged. Thus, LP tokens remain unchanged when utilizing excess `SOL` for withdrawals, enabling users to withdraw `SOL` without burning LP tokens, thereby maintaining an inflated share.

© 2025 Otter Audits LLC. All Rights Reserved.

Remediation

Reduce `deposited_tokens` proportionally. Even when utilizing excess `SOL`, deduct `deposited_tokens` based on the current LP share value.

© 2025 Otter Audits LLC. All Rights Reserved.

Inconsistencies in Eject User Instruction OS-ATF-SUG-01

Description

1. Explicitly check if the user has already been ejected instead of relying on setting `user_account.deposited_lamports` implicitly to zero to improve clarity and ensure that the contract behaves predictably.
2. `EjectUser` may fail if the user's lamports are less than the minimum required rent (`rent_min`) needed to maintain their account. Ensuring the protocol adds enough lamports to the user's account to allow ejection while keeping the account rent-exempt.

Remediation

Implement the above-mentioned suggestions.

© 2025 Otter Audits LLC. All Rights Reserved.

Code Maturity OS-ATF-SUG-02

Description

1. Currently, the `vault_authority` may be changed in a single-step process via `auth::change_vault_authority_inner`, lacking a confirmation step. As a result, once the transaction is submitted, the authority change is irreversible. This gives rise to the possibility of denial of service when the current authority inadvertently sends an unintended input as a parameter while executing a change of authority. Utilize a two-step process where the new authority explicitly accepts ownership.

   >_ programs/stake-vault/src/instructions/auth.rs RUST

   ```rust
   pub fn change_vault_authority_inner(
       ctx: Context<ChangeVaultAuthority>,
       new_authority: Pubkey,
   ) -> Result<()> {
       let vault_state = &mut ctx.accounts.vault_state;
       vault_state.vault_authority = new_authority;

       msg!(
           "Updated vault authority to: {:?}",
           vault_state.vault_authority
       );

       Ok(())
   }
   ```

2. Implement an instruction that allows the vault authority to update the `minimum_lockup_period_seconds` field in the `VaultState` structure within `Vault`.
3. Utilize `deposit_sol_with_slippage` and `withdraw_sol_with_slippage` instead of `deposit_sol` and `withdraw_sol` when calling the Stake Pool program via Cross-Program Invocation (CPI). This will enable users to specify an acceptable range of slippage, ensuring their transactions will only be processed if the amount of SOL they are depositing or withdrawing is within the specified slippage limit. This prevents users from facing unexpected losses or receiving fewer tokens than expected due to sudden changes in pool values.

Remediation

Incorporate the above-listed suggestions.

© 2025 Otter Audits LLC. All Rights Reserved.

Code Redundancy

OS-ATF-SUG-03

Description

1. In `Distribute::ViewAvailableExcess`, `vault_state` and `vault_token_account` do not need to be mutable, and as a result, `mut` may be removed.

   >_ programs/stake-vault/src/instructions/distribute.rs RUST

   ```rust
   #[derive(Accounts)]
   pub struct ViewAvailableExcess<'info> {
       /// The vault's state account storing deposit tracking
       #[account(mut, seeds = [VAULT_SEED], bump)]
       vault_state: Account<'info, VaultState>,

       /// The vault's token account holding staked tokens
       #[account(
           mut,
           associated_token::mint = jito_mint,
           associated_token::authority = vault_state
       )]
       pub vault_token_account: Account<'info, TokenAccount>,
       [...]
   }
   ```

2. In `DepositSol` instruction, `referrer_token_account` appears to be the same as `vault_token_account`, making it redundant. It may be removed from the instruction context.
3. `WithdrawSol` and `EjectUser` share common functionality that should be extracted into separate functions to reduce redundancy.

Remediation

Remove the redundant code instances listed above.

© 2025 Otter Audits LLC. All Rights Reserved.

A — Vulnerability Rating Scale

We rated our findings according to the following scale. Vulnerabilities have immediate security implications. Informational findings may be found in the General Findings.

CRITICAL

Vulnerabilities that immediately result in a loss of user funds with minimal preconditions.

Examples:

- Misconfigured authority or access control validation.
- Improperly designed economic incentives leading to loss of funds.

HIGH

Vulnerabilities that may result in a loss of user funds but are potentially difficult to exploit.

Examples:

- Loss of funds requiring specific victim interactions.
- Exploitation involving high capital requirement with respect to payout.

MEDIUM

Vulnerabilities that may result in denial of service scenarios or degraded usability.

Examples:

- Computational limit exhaustion through malicious input.
- Forced exceptions in the normal user flow.

LOW

Low probability vulnerabilities, which are still exploitable but require extenuating circumstances or undue risk.

Examples:

- Oracle manipulation with large capital requirements and multiple transactions.

INFO

Best practices to mitigate future security risks. These are classified as general findings.

Examples:

- Explicit assertion of critical internal invariants.
- Improved input validation.

© 2025 Otter Audits LLC. All Rights Reserved.

B — Procedure

As part of our standard auditing procedure, we split our analysis into two main sections: design and implementation.

When auditing the design of a program, we aim to ensure that the overall economic architecture is sound in the context of an on-chain program. In other words, there is no way to steal funds or deny service, ignoring any chain-specific quirks. This usually requires a deep understanding of the program's internal interactions, potential game theory implications, and general on-chain execution primitives.

One example of a design vulnerability would be an on-chain oracle that could be manipulated by flash loans or large deposits. Such a design would generally be unsound regardless of which chain the oracle is deployed on.

On the other hand, auditing the program's implementation requires a deep understanding of the chain's execution model. While this varies from chain to chain, some common implementation vulnerabilities include reentrancy, account ownership issues, arithmetic overflows, and rounding bugs.

As a general rule of thumb, implementation vulnerabilities tend to be more "checklist" style. In contrast, design vulnerabilities require a strong understanding of the underlying system and the various interactions: both with the user and cross-program.

As we approach any new target, we strive to comprehensively understand the program first. In our audits, we always approach targets with a team of auditors. This allows us to share thoughts and collaborate, picking up on details that others may have missed.

While sometimes the line between design and implementation can be blurry, we hope this gives some insight into our auditing procedure and thought process.

© 2025 Otter Audits LLC. All Rights Reserved.